Exp. No: 6

FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS

Code:

```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

tf.keras.utils.disable_interactive_logging()


# 2. Feedforward Neural Network (FNN)

# 2.1 Load Dataset
(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) =
fashion_mnist.load_data()

print("Original FNN training data shape:", x_train_fnn.shape)
print("Original FNN test data shape:", x_test_fnn.shape)


# 2.2 Flatten Images
x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)
```

```
# 2.3 Normalize Pixel Values
x_train_fnn_norm = x_train_fnn_flat / 255.0
x_test_fnn_norm = x_test_fnn_flat / 255.0

print(
  "Flattened & Normalized FNN training data shape:",
  x_train_fnn_norm.shape
)

print(
  "Flattened & Normalized FNN test data shape:",
  x_test_fnn_norm.shape
)



# 2.4 Build FNN Model
model_fnn = keras.Sequential([
  layers.Dense(128, activation="relu", input_shape=(784,)),
  layers.Dropout(0.2),
  layers.Dense(64, activation="relu"),
  layers.Dense(10, activation="softmax")
])



# 2.5 Compile FNN Model
model_fnn.compile(
  optimizer="adam",
  loss="sparse_categorical_crossentropy",
  metrics=["accuracy"]
)

print("\nFNN Model Summary:")
model_fnn.summary()



# 2.6 Train FNN Model
print("\nTraining FNN Model...")
```

```
history_fnn = model_fnn.fit(
  x_train_fnn_norm,
  y_train_fnn,
  epochs=10,
  validation_split=0.1,
  verbose=1
)


# 2.7 Evaluate FNN Model
print("\nEvaluating FNN Model...")

loss_fnn, accuracy_fnn = model_fnn.evaluate(
  x_test_fnn_norm,
  y_test_fnn,
  verbose=0
)

print(f"FNN Test Loss: {loss_fnn:.4f}")
print(f"FNN Test Accuracy: {accuracy_fnn:.4f}")


# 2.8 FNN Predictions
y_pred_fnn = np.argmax(
  model_fnn.predict(x_test_fnn_norm, verbose=0),
  axis=-1
)


# 2.9 FNN Classification Report
print("\nFNN Classification Report:")
print(
  classification_report(
    y_test_fnn,
    y_pred_fnn
  )
)
```

```
# 2.10 FNN Confusion Matrix
cm_fnn = confusion_matrix(
  y_test_fnn,
  y_pred_fnn
)

plt.figure(figsize=(10, 8))

sns.heatmap(
  cm_fnn,
  annot=True,
  fmt="d",
  cmap="Blues",
  cbar=False
)

plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()



# 2.11 Plot FNN Accuracy and Loss
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(
  history_fnn.history["accuracy"],
  label="Training Accuracy"
)

plt.plot(
  history_fnn.history["val_accuracy"],
  label="Validation Accuracy"
)
```

```
plt.title("FNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)

plt.plot(
  history_fnn.history["loss"],
  label="Training Loss"
)

plt.plot(
  history_fnn.history["val_loss"],
  label="Validation Loss"
)

plt.title("FNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# 3. Convolutional Neural Network (CNN)

# 3.1 Load Dataset
(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) =
mnist.load_data()

print("\nOriginal CNN training data shape:", x_train_cnn.shape)
print("Original CNN test data shape:", x_test_cnn.shape)
```

```
# 3.2 Reshape Images
x_train_cnn = x_train_cnn.reshape(
  x_train_cnn.shape[0],
  28,
  28,
  1
)

x_test_cnn = x_test_cnn.reshape(
  x_test_cnn.shape[0],
  28,
  28,
  1
)



# 3.3 Normalize Pixel Values
x_train_cnn = x_train_cnn.astype("float32") / 255.0
x_test_cnn = x_test_cnn.astype("float32") / 255.0

print(
  "Reshaped & Normalized CNN training data shape:",
  x_train_cnn.shape
)

print(
  "Reshaped & Normalized CNN test data shape:",
  x_test_cnn.shape
)

num_classes_cnn = 10

# 3.4 Build CNN Model
model_cnn = keras.Sequential([
  layers.Conv2D(
```

```
    32,
    (3, 3),
    activation="relu",
    input_shape=(28, 28, 1)
  ),
  layers.MaxPooling2D((2, 2)),
  layers.Conv2D(
    64,
    (3, 3),
    activation="relu"
  ),
  layers.MaxPooling2D((2, 2)),
  layers.Flatten(),
  layers.Dense(
    128,
    activation="relu"
  ),
  layers.Dropout(0.5),
  layers.Dense(
    num_classes_cnn,
    activation="softmax"
  )
])


# 3.5 Compile CNN Model
model_cnn.compile(
  optimizer="adam",
  loss="sparse_categorical_crossentropy",
  metrics=["accuracy"]
)

print("\nCNN Model Summary:")
model_cnn.summary()

# 3.6 Train CNN Model
print("\nTraining CNN Model...")
```

```
history_cnn = model_cnn.fit(
  x_train_cnn,
  y_train_cnn,
  epochs=10,
  validation_split=0.1,
  verbose=1
)



# 3.7 Evaluate CNN Model
print("\nEvaluating CNN Model...")

loss_cnn, accuracy_cnn = model_cnn.evaluate(
  x_test_cnn,
  y_test_cnn,
  verbose=0
)

print(f"CNN Test Loss: {loss_cnn:.4f}")
print(f"CNN Test Accuracy: {accuracy_cnn:.4f}")



# 3.8 CNN Predictions
y_pred_cnn = np.argmax(
  model_cnn.predict(x_test_cnn, verbose=0),
  axis=-1
)



# 3.9 CNN Classification Report
print("\nCNN Classification Report:")

print(
  classification_report(
    y_test_cnn,
```

```
    y_pred_cnn
  )
)

# 3.10 CNN Confusion Matrix
cm_cnn = confusion_matrix(
  y_test_cnn,
  y_pred_cnn
)

plt.figure(figsize=(10, 8))

sns.heatmap(
   cm_cnn,
   annot=True,
   fmt="d",
   cmap="Blues",
   cbar=False
)

plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()


# 3.11 Plot CNN Accuracy and Loss
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(
   history_cnn.history["accuracy"],
   label="Training Accuracy"
)
```

```
plt.plot(
  history_cnn.history["val_accuracy"],
  label="Validation Accuracy"
)

plt.title("CNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)



plt.subplot(1, 2, 2)

plt.plot(
  history_cnn.history["loss"],
  label="Training Loss"
)

plt.plot(
  history_cnn.history["val_loss"],
  label="Validation Loss"
)

plt.title("CNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 4. Sample CNN Predictions

class_names_mnist = [
  str(i) for i in range(10)
]
```

```
plt.figure(figsize=(10, 10))

for i in range(25):
  plt.subplot(5, 5, i + 1)

  plt.xticks([])
  plt.yticks([])
  plt.grid(False)

  plt.imshow(
    x_test_cnn[i].reshape(28, 28),
    cmap=plt.cm.binary
  )

  true_label = y_test_cnn[i]
  predicted_label = y_pred_cnn[i]

  color = (
    "green"
    if true_label == predicted_label
    else "red"
  )

  plt.xlabel(
    f"True: {class_names_mnist[true_label]}\n"
    f"Pred: {class_names_mnist[predicted_label]}",
    color=color
  )

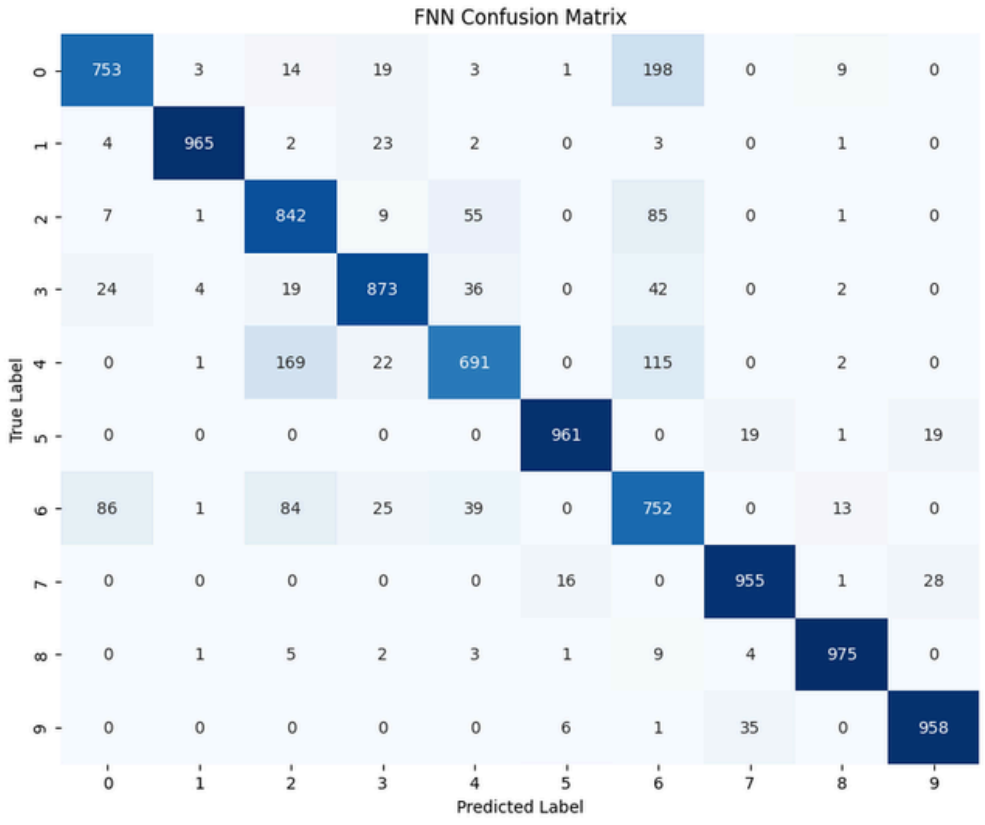
plt.suptitle(
  "Sample CNN Predictions "
  "(Green: Correct, Red: Incorrect)",
  y=1.02,
  fontsize=16
)
```

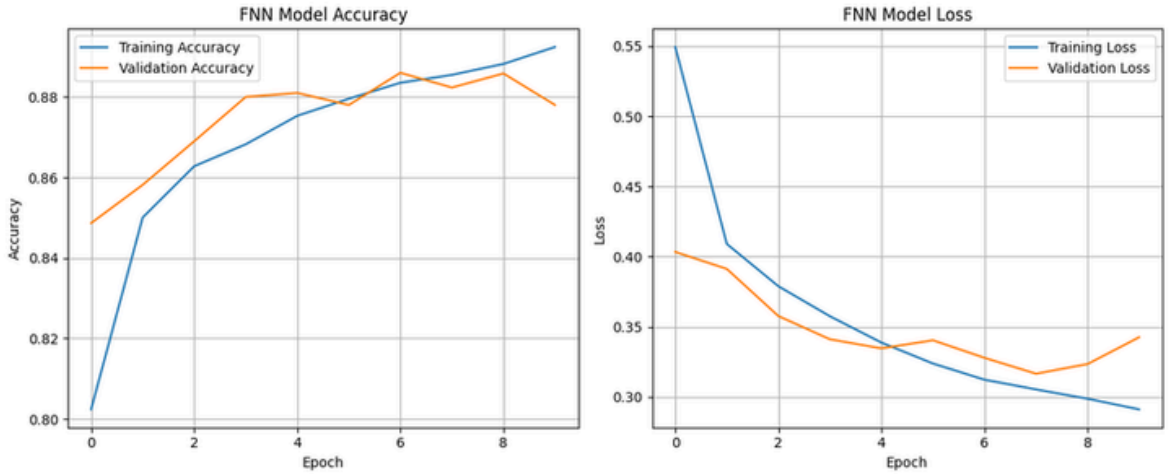
```
plt.tight_layout(
 rect=[0, 0, 1, 0.98]
)

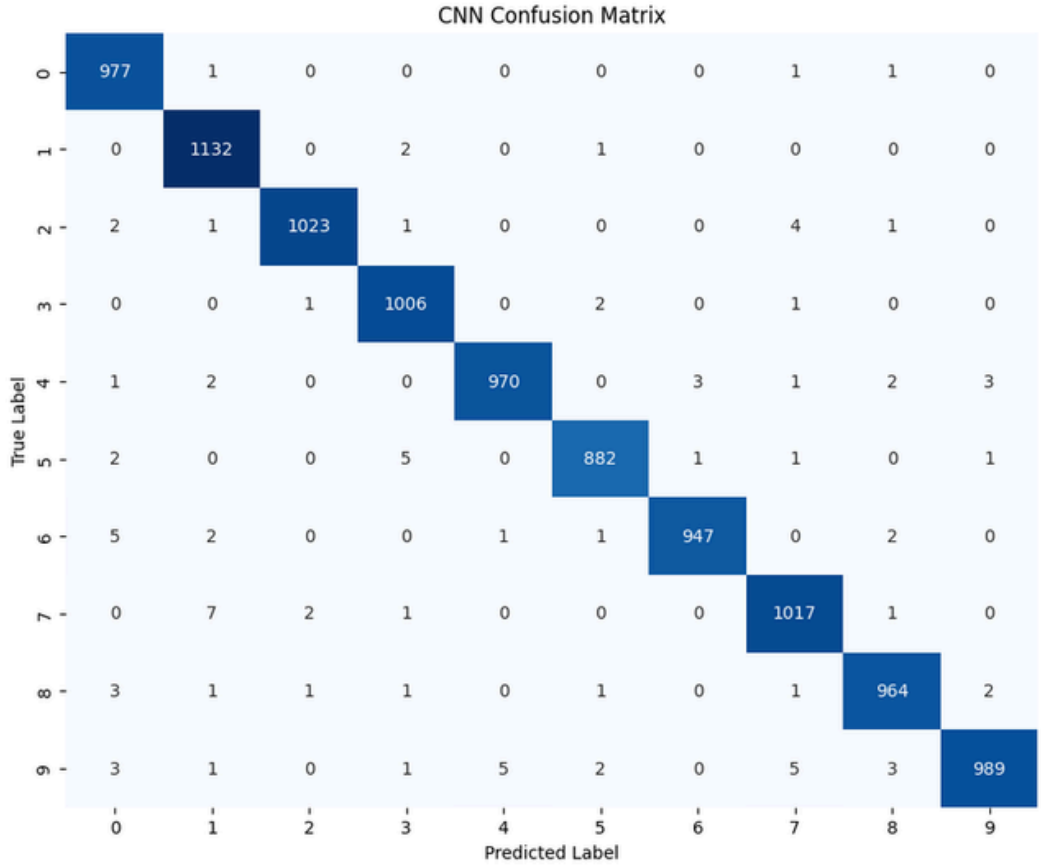
plt.show()

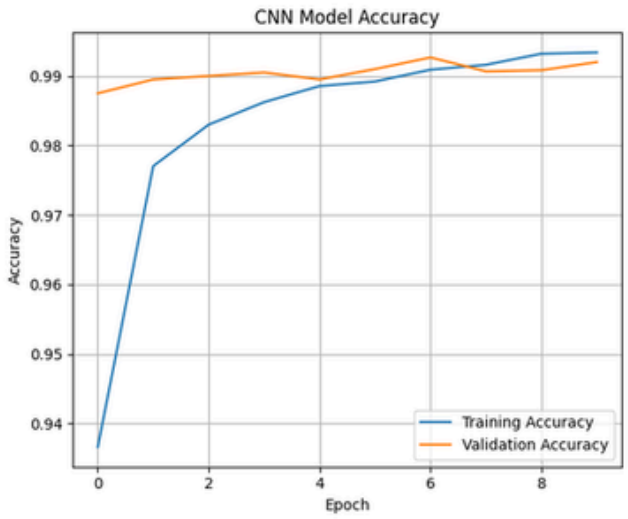
# 5. Final Output
print("\nExperiment completed successfully!")
```

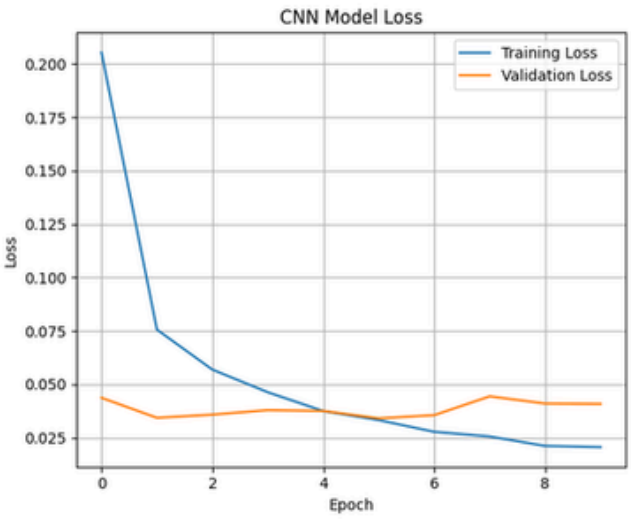
Output:






CNN Confusion Matrix
True Label
Predicted Label
0 977 1 0 0 0 0 0 1 1 0
1 0 1132 0 2 0 1 0 0 0 0
2 2 1 1023 1 0 0 0 4 1 0
3 0 0 1 1006 0 2 0 1 0 0
4 1 2 0 0 970 0 3 1 2 3
5 2 0 0 5 0 882 1 1 0 1
6 5 2 0 0 1 1 947 0 2 0
7 0 7 2 1 0 0 0 1017 1 0
8 3 1 1 1 0 1 0 1 964 2
9 3 1 0 1 5 2 0 5 3 989
0 1 2 3 4 5 6 7 8 9


CNN Model Accuracy
Accuracy
Epoch
Training Accuracy
Validation Accuracy
0.99
0.98
0.97
0.96
0.95
0.94
0
2
4
6
8


CNN Model Loss
Loss
Epoch
Training Loss
Validation Loss
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0
2
4
6
8

Sample CNN Predictions (Green: Correct, Red: Incorrect)

